

Chapter 1

Understanding Cloud Computing

1.1 Introduction to Cloud Computing

1.1.1 Introduction to cloud computing

1.1.2 The power of cloud

- Vertical and Horizontal Scaling
- Pay as you go pricing
- Speed
- Reliability
- Security

1.1.3 Cloud service models

- IaaS: owning car
- PaaS: ridesharing
- SaaS: taking a bus
- FaaS: Function as a Service, auth and transaction
- XaaS: Anything as a Service, covers everything

1.2 Cloud Deployment

1.2.1 Cloud Deployment models

- Private cloud: uses virtualization for on demand resources and can be located off-premises, direct control over resources available, and how data is stored
- Public cloud: internet accessible for general public, no access to data center and hardware
- Hybrid cloud: mix, for seasonal spikes or cloud bursting
- Multicloud : combine different CSP
- Community cloud: shared by specific community with common interest

1.2.2 Regulations on the cloud

Different region have different view on how data should be managed

- General Data Protection Regulation (GDPR): EU, explicit consent, data breaches notification, encrypted, anonymized PI, can't leave EU borders unless guarantee same level of protection, fine of 20 mil or 4% of worldwide revenue whichever is greater
- https://commission.europa.eu/law/law-topic/data-protection/data-protection-explained_en
- Brazil's Lei Geral de Protecao de Dados (LGPD)
- California's Consumer Privacy Act (CCPA)
- USA' Health Insurance Portability and Accountability Act (HIPPA)
- Japan's Act on Protection of Personal Information
- Thailand Personal Data Protection Act (PDPA)
- Canada's Personal Information and Electronic Documents Act (PIPEDA)

1.2.3 Cloud computing roles

- Data scientist: run computationally expensive analysis on cloud
- Machine Learning scientist: train and deploy ML models on the cloud
- Data engineer: build pipelines on the cloud to ingest, transform, and store big data
- Data analyst: access data on the cloud via BI tools
- Cloud architect: solution architect for the cloud
- Cloud engineer: role for anything technical on cloud
- DevOps engineer: Software **D**evelopment + IT **O**perations, IaC
- Security engineer: test and assess security of data on cloud

1.3 Cloud Providers and Case Studies

1.3.1 An overview of providers

- AWS, Azure, GCP, Alibaba Cloud, salesforce, IBM Cloud, Oracle cloud, tencent cloud
- Best cloud provider meets company needs

1.3.2 Amazon Web Services

- launched in 2006
- more than 175 services in computing, storage, analytics, security and ML
- >31% market share
- EC2 for computation, RDS for database, S3 for storage
- Data Services: Kinesis for real time data and analytics, Redshift for analytics data warehousing, Sagemaker for predictive analytics
- AWS customers: Disney+, Deloitte, verizon

- Case Study: NerdWallet takes too long to deploy ML model, use Sagemaker to reduce training times to days and training cost by 75%
- aws.amazon.com/solutions/case-studies

1.3.3 Microsoft Azure

- launched in 2008
- Integration with Microsoft products
- >24% market shared
- Azure Blob storage for file storage, Azure Virtual Machine for computation, Azure SQL Database for databases,
- Data Services: Microsoft Fabric is SaaS for enterprise use that brings together different Microsoft solutions which covers everything from data movement to data science, realtime analytics, and BI, Data Lake storage for storing data before cleaning, Stream Analytics for real-time analytics and Machine Learning, to train and deploy ML models
- Azure customers: SIEMENS, L'ORÉAL, THE WORLD BANK
- Case Study: Ottawa Hospital needs cost effective and secure disaster recovery solution, Microsoft IaaS, Azure storage for medical imaging data, Azure Site Recovery for recovery processes automatically when a data loss incident occurs saving 50% on disaster recovery cost and compliant infrastructure
- customers.microsoft.com

1.3.4 Google Cloud

- launched in 2010
- Google Cloud Anthos for hybrid multi-cloud solution in 2019
- >11% market share
- Google Cloud Storage, Google Cloud Compute Engine, Google cloud SQL
- Data Services: BigQuery for data warehouse, Data flow for batch and stream data processing, AutoML for ML model training and development
- Google Cloud Customers : HSBC, PayPal, vodafone
- Case Study: LUSH needs availability and stability during peak loads migrated entire global infra to Google cloud, Google cloud Compute Engine to test and provision in migrating services with data in Google Cloud SQL reducing 40% in hosting cost
- cloud.google.com/customer/lush/

Chapter 2

Introduction to Python for Developers

2.1 Introduction to Python

2.1.1 What is Python?

- General-purpose programming language
- Free and open source

The `print` function

- Built-in Python function
- Displays code results
- Used for testing
- Can use either single or double quotations

```
print("Hello, world!")  
# Hello, world!  
print('Hello, world!')  
# Hello, world!
```

Documenting with comments

- Comments explain code
- Starts with `#` symbol

```
# This is a code comment  
# The below is a print function  
print("Hello, world!")
```

2.1.2 Python variables, strings, and integers

Data types

- The kind of value
 - Number
 - Text
 - True/False

- Python assign data types automatically

String data types and variables

- **Strings:** text values
- Can contain letters, numbers, and punctuation
- Use single or double quotes
- Double quotes for words with apostrophes
- use lowercase and underscores for spaces i.e. `ingredient_name`

```
ingredient_name = "Tomatoes"
```

Integer data types and variables

- **Integer:** whole numbers

```
ingredient_quantity = 2
```

Printing and adjusting variables

```
print(ingredient_name)
# Tomatoes
print(ingredient_quantity)
# 2
ingredient_quantity = 1
print(ingredient_quantity)
# 1
```

2.1.3 Common data types

Decimal data types

- **Floats:** numbers with decimal, e.g. 1.5

```
new_ingredient_quantity = 1.5
```

Boolean data types

- **Boolean:** True or False
- Great for storing yes/no information
- Capitalized
- No quotation marks

```
is_in_stock = True
is_in_stock = False
```

The type function

- Data types:
 - Strings
 - Integers
 - Float

- Booleans

```
type()
print(type(is_in_stock))
# <class 'bool'>
```

- Tells us the data type

Operators

Symbols called operators to perform computations

Arithmetic operators

- `+` for addition
- `-` for subtraction
- `*` for multiplication
- `/` for division

Numbers Work like regular math

```
print(2 + 1.5)
# 3.5
```

Strings Only `+` and `*` works

```
print("Hi" + "There")
# "HiThere"
print("Hi" * 3)
# "HiHiHi"
```

2.2 Working with Data Types

2.2.1 Working with strings

Multi-line strings

```
# Create a string variable over multiple lines
recipe_instructions = """1. Bring a large pot of salted water to boil and cook pasta
2. Heat olive oil in a pan and sauté minced garlic until fragrant
3. Add chopped tomatoes and simmer for 10 minutes
4. Toss cooked pasta with tomato sauce and fresh basil leaves
"""
```

- `"""text"""`: Multi-line strings
 - Enhance readability
 - Used for documentation
 - Longer text such as instructions or error messages

Methods

- Method = a function that is only available to a specific data type
- `str` Methods
 - Standardizing input, or transforming text

```
# Calling a string method
string_variable.method()
```

Replacing parts of strings

```
welcome_message = "Welcome to the recipe scaler, George"
```

```
welcome_message = welcome_message.replace("George", "John")
print(welcome_message)
# Welcome to the recipe scaler, John
```

Changing case

- Standardize user inputs like emails

```
ingredient_name = "Basil Leaves"
#Convert to lowercase
ingredient_name = ingredient_name.lower()
print(ingredient_name)
# basil leaves
```

```
#Change to uppercase
ingredient_name = ingredient_name.upper()
print(ingredient_name)
# BASIL LEAVES
```

2.2.2 Lists

- **List** = store multiple values in a single variables
- Can contain any combination of data types
- Lists are ordered and indexed

```
ingredient_one = "pasta"
ingredient_two = "tomatoes"
ingredient_three = "garlic"
ingredient_four = "basil"
ingredient_five = "olive oil"
ingredient_six = "salt"
```

```
# List of ingredients
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]
```

```
# List of ingredients using variables as values
ingredients = [ingredient_one, ingredient_two, ingredient_three, ingredient_four, ingredient_five, ingredient_six]
```

Checking the data type

```
# Check the data type of a list
print(type(ingredients))
# <class 'list'>
```

Accessing elements of a List

- List = []
- Access an element = a_list[index]

```
# ['pasta', 'tomatoes', 'garlic', 'basil', 'olive oil', 'salt']
```

```
# Get the value of the first index
```

```
print(ingredients[0])
```

```
# pasta
```

```
# Get the fourth element
```

```
print(ingredients[3])
```

```
# basil
```

Finding the last element in a list

```
# Get the last element of a list
```

```
print(ingredients[5])
```

```
# salt
```

```
# Get the last element of a list for unknown length
```

```
print(ingredient[-1])
```

```
# salt
```

Accessing multiple elements

-

2.2.3 Dictionaries

2.2.4 Sets and tuples

2.3 Control flows and loops

2.3.1 Conditional statements and operators

2.3.2 For loops

2.3.3 While loops

2.3.4 Building a workflow